TensorConversion.h
TensorConversion.c

A Beginner's Line-by-Line Guide to Tensor Type Conversion

On-Device Training Framework — Master's Thesis Project

What you will learn in this PDF:

- What quantization types exist and how they differ
- How the conversionFunction_t function pointer typedef works
- Every conversion function explained line by line
- How the 6×6 conversionMatrix dispatch table works
- How convertTensor() picks the right function automatically
- What requantization means and the difference between dynamic vs pre-set scale

1 — What Does TensorConversion Do?

Tensors in this framework can store numbers in different formats — floating-point, plain integer, quantized integer with a scale factor, and so on. Each format is called a **quantization type** (`qtype_t`). TensorConversion provides one entry-point function, `convertTensor()`, that converts the data inside any tensor from one type to any other type. You never need to call a specific conversion function yourself — just call `convertTensor()` and it figures out which converter to use.

The six quantization types:

qtype_t name	Meaning
`INT32`	Plain 32-bit signed integer. No scale. Range: −2,147,483,648 to +2,147,483,647.
`FLOAT32`	32-bit IEEE 754 floating-point. Widest range and precision. Used during training.
`SYM_INT32`	Symmetric fixed-point: stores int32 mantissa values, plus a float scale. Real value = mantissa × scale. Zero-point is always 0 (symmetric around 0).
`SYM`	General symmetric quantization to N bits. Scale only.
`ASYM`	Asymmetric N-bit quantization. Has both scale and a zeroPoint offset.
`BOOL`	Single bit per element (true/false). Packed: 8 elements per byte.

★ SYM_INT32 is the workhorse for this project's Fixed-Point Quantization (FQT) training. Weights and activations are stored as SYM_INT32 during quantized forward/backward passes.

2 — TensorConversion.h — The Header File

The header declares everything callers need: the function pointer type, the public functions, and the external matrix. Implementation details stay in the .c file.

Line-by-line walkthrough:

`#ifndef TENSOR_CONVERSION_H` `#define TENSOR_CONVERSION_H`	Include guard: prevents this file from being included twice. If it was already included, everything inside is skipped.
`#include "Tensor.h"`	Pulls in tensor_t, qtype_t, and all shape/quantization structs. We need tensor_t because our functions take tensor_t* arguments.
`typedef void (*conversionFunction_t)(tensor_t *inputTensor,` `tensor_t *outputTensor);`	Declares a FUNCTION POINTER TYPE. Any function that takes two tensor_t* arguments and returns void matches this type. The 6×6 dispatch table stores pointers of this type.

■ **Function pointer crash course:** A normal variable stores data. A function pointer stores the *address of a function*. You can call it later without knowing at compile time which function it is. `conversionFunction_t fn = convertFloatTensorToInt32Tensor; fn(a, b);` — here fn holds the address; calling fn(a,b) executes that function.

`void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor);`	THE main entry point. Call this to convert any tensor to any other type. It auto-selects the right converter.
`void requantSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor);`	SYM_INT32 → SYM_INT32 requantization. Computes a FRESH scale from the actual data range (two-pass). Good when you do not know the target scale in advance.
`void requantSymInt32TensorToScale(tensor_t *inputTensor, tensor_t *outputTensor);`	SYM_INT32 → SYM_INT32 requantization into a PRE-SET scale. The caller writes the desired scale into the output tensor's qConfig BEFORE calling this. One-pass. Saturates if values overflow.
`char *quantTypeToString(qtype_t t);`	Helper: converts a qtype_t enum value to a human-readable string like "FLOAT32" or "SYMINT32". Used in error messages.
`extern conversionFunction_t conversionMatrix[6][6];`	Declares (does not define) the 6×6 table of function pointers. The actual table lives in TensorConversion.c. extern means "it exists somewhere — look it up at link time".
`#endif // TENSOR_CONVERSION_H`	Closes the include guard.

3 — TensorConversion.c — Includes and Helper Functions

Includes:

`#define SOURCE_FILE "TENSOR_CONVERSION"`
`#include`
`#include`
`#include`
`#include "Common.h"`
`#include "DTypes.h"`
`#include "MinMax.h"`
`#include "Tensor.h"`
`#include "TensorConversion.h"`
`#include "math.h"`
	SOURCE_FILE – used by PRINT_ERROR to say where a crash came from. **stdio/stdlib/string.h** – printf, exit(), memcpy/memset. **Common.h** – PRINT_ERROR macro. **DTypes.h** – read/write byte helpers. **MinMax.h** – findAbsMaxFloat(), findMinFloat(), findMaxFloat(). **Tensor.h** – tensor_t struct and size calculators. **math.h** – powf(), fabsf().

Helper Function 1: zeroTensorData()

```
void zeroTensorData(tensor_t *tensor) {
size_t numberOfElements =
calcNumberOfElementsByTensor(tensor);
size_t bytesPerElement =
calcBytesPerElement(tensor->quantization);
memset(tensor->data, 0,
numberOfElements * bytesPerElement);
}
```

What it does: fills the tensor's data buffer with zero bytes. **calcNumberOfElementsByTensor** – multiplies all shape dimensions. **calcBytesPerElement** – returns sizeof(float) for FLOAT32, sizeof(int32_t) for INT32/SYM_INT32, etc. **memset(ptr, 0, n)** – writes n zero bytes starting at ptr. Used before writing a conversion result to make sure no garbage bits remain.

Helper Function 2: copyDimsAndSparsityToTensor()

```
void copyDimsAndSparsityToTensor(
tensor_t *inputTensor,
tensor_t *outputTensor) {
outputTensor->shape = inputTensor->shape;
if (inputTensor->sparsity) {
memcpy(outputTensor->sparsity,
inputTensor->sparsity,
sizeof(sparsity_t));
}
}
```

What it does: after converting the numeric data, make sure the output tensor has the same shape and sparsity info as the input. **shape:** pointer copy — both tensors share the same shape_t. That is safe because conversions do not change shape. **sparsity:** NULL-checked, then deep-copied with memcpy so the structs are independent. Called at the end of most conversion functions.

4 — INT32 Conversions

These three functions convert a tensor whose data is stored as plain 32-bit integers (no scale, no quantization config) into other formats.

convertInt32TensorToFloatTensor()

```
void convertInt32TensorToFloatTensor(
tensor_t *inputTensor,
tensor_t *outputTensor) {
size_t n =
calcNumberOfElementsByTensor(inputTensor);
int32_t inputData[n];
float outputData[n];
readBytesAsInt32Array(n, inputTensor->data,
inputData);
zeroTensorData(outputTensor);
for (size_t i = 0; i < n; i++) {
outputData[i] = (float)inputData[i];
}
writeFloatArrayToByteArray(n, outputData,
outputTensor->data);
copyDimsAndSparsityToTensor(inputTensor,
outputTensor);
}
```

Step 1: Count elements (n). **Step 2:** Read raw bytes as int32 values into a stack array. **Step 3:** Zero the output buffer (clean slate). **Step 4:** Cast each int32 to float: (float)inputData[i]. e.g. 42 → 42.0f. **Step 5:** Write float values back as raw bytes into output. **Step 6:** Copy shape + sparsity.

✔ INT32 → FLOAT32: simple C cast. Value 42 stays 42.0f. No scale involved.

convertInt32TensorToSymInt32Tensor()

```
void convertInt32TensorToSymInt32Tensor(
tensor_t *inputTensor,
tensor_t *outputTensor) {
size_t n =
calcNumberOfElementsByTensor(inputTensor);
symInt32QConfig_t *outQC =
outputTensor->quantization->qConfig;
outQC->scale = 1;
memcpy(outputTensor->data, inputTensor->data,
n * sizeof(int32_t));
}
```

Data: bytes are identical — int32 and SYM_INT32 both use 4 bytes per element. memcpy just duplicates the buffer. **Scale:** set to 1 because SYM_INT32 real_value = mantissa × scale, and we want real_value == mantissa, so scale=1. Very fast: one memcpy, no arithmetic.

convertInt32TensorToAsymTensor()

```
void convertInt32TensorToAsymTensor(...) {
int32_t min = findMinInt32(...);
int32_t max = findMaxInt32(...);
int32_t qMax = pow(2, qBits) - 1;
float scale = (float)(max-min) / (float)qMax;
int16_t zeroPoint =
roundByMode(min/scale, roundingMode);
for each element:
out[i] = clamp(x/scale - zeroPoint, 0, qMax-1);
byteConversion(..., 32, ..., qBits, n);
}
```

Asymmetric quantization maps a real range [min,max] to [0, 2^qBits-1]. scale: how many real units per quantized step. **zeroPoint:** which quantized value represents real-value 0. **Formula:** q = clamp(x/scale – zeroPoint, 0, qMax). **byteConversion:** packs 32-bit intermediates down to qBits-bit output (e.g. 8-bit).

5 — FLOAT32 Conversions

These functions take a FLOAT32 tensor and produce various quantized outputs. FLOAT32 → SYM_INT32 is the most important for FQT forward passes.

convertFloatTensorToInt32Tensor()

```
outputData[i] = (int32_t)inputData[i];
```

Simply truncates the fractional part: 3.7f → 3. Fast, but lossy for non-integer floats.

convertFloatTensorToSymInt32Tensor() ← most important

```
float absMax = findAbsMaxFloat(inputTensor->data,
n);
uint8_t qMaxBits = symInt32QC->qMaxBits;
const float qMax = powf(2, qMaxBits-1) - 1;
const float qMin = -powf(2, qMaxBits-1);
scale = (absMax==0) ? 1.f : absMax/qMax;
outputSymInt32QC->scale = scale;
outputInt32[i] = roundByMode(
clamp(inputFloat[i]/scale, qMin, qMax),
roundingMode);
```

Goal: pack float values into int32 mantissas so that mantissa×scale ≈ original float. **absMax:** the largest absolute value — determines the scale so the biggest value maps to ±qMax. **qMaxBits=8:** qMax=127, qMin=-128 (like int8). **scale = absMax/qMax**: e.g. absMax=0.5, qMax=127 → scale=0.00394. **clamp** prevents out-of-range values (shouldn't happen with absMax-derived scale). This is the KEY function for quantization-aware training (FQT).

★ **Example:** Weights: [-0.5, 0.2, 0.4, -0.1], qMaxBits=8. absMax=0.5, qMax=127, scale=0.5/127≈0.00394. Mantissas: [-127, 51, 102, -25]. Stored in int32 array. To recover: mantissa × 0.00394 ≈ original.

convertFloatTensorToSymTensor()

```
float qMax = powf(2, outputSymQConfig->qBits);
float scale = (min - max) / qMax;
outputs[i] = clamp(inputs[i]/scale, 0, qMax-1);
```

Symmetric N-bit quantization. Note: range is [min,max], symmetric around 0. Marked with a comment "I DON'T HAVE TO IMPLEMENT SYM CONVERSIONS!" — incomplete/placeholder.

convertFloatTensorToAsymTensor()

Same logic as convertInt32TensorToAsymTensor but starting from float values. Comment says it 'should not be needed/used' — exists for completeness.

6 — SYM_INT32 Conversions

SYM_INT32 tensors store a mantissa (int32) plus a float scale. Converting them to/from other types requires multiplying or dividing by the scale.

extractInt32TensorFromSymInt32Tensor()

```
// Important: Scale is IGNORED!
memcpy(outputTensor->data, inputAsInt32,
numberOfElements * sizeof(int32_t));
```

Just copies the raw int32 mantissas, dropping the scale. The output tensor is plain INT32. Used when you only need the integer part without dequantizing.

convertSymInt32TensorToFloat32Tensor()

```
float scale = inputSymInt32QConfig->scale;
for (size_t i = 0; i < n; i++) {
output[i] = (float)inputAsInt32[i] * scale;
}
```

Dequantization: multiply each mantissa by the scale to recover the original float. If mantissa[i]=102 and scale=0.00394, then output[i]=102×0.00394≈0.402. This is the inverse of convertFloatTensorToSymInt32Tensor.

requantSymInt32Tensor() — Dynamic Rescaling (Two-Pass)

■ Scenario: you have a SYM_INT32 tensor with scale A, but you want to fit the same values into a SYM_INT32 tensor with a FRESH scale B computed from the data. This happens after accumulating gradients (they may have a different range than weights).

```
float inScale = inputSymInt32QC->scale;
// Latch BEFORE writing output (alias-safe).
// Pass A: find absMax of dequantized values
float absMax = 0.f;
for (size_t i = 0; i < n; i++) {
float dq = fabsf((float)inputInt32[i] * inScale);
if (dq > absMax) absMax = dq;
}
scale = (absMax==0) ? 1.f : absMax/qMax;
outputSymInt32QC->scale = scale; // write new
scale
// Pass B: requantize into new scale
for (size_t i = 0; i < n; i++) {
outputInt32[i] = roundByMode(
clamp((inputInt32[i]*inScale)/scale,
qMin, qMax),
roundingMode);
}
```

Why two passes? Pass A reads all values to find the range (absMax). Only then can we compute the optimal scale. Pass B uses that scale to convert every value. **In-place safe:** even if input==output (same pointer), pass A only reads, then pass B does same-index read-then-write — no element is overwritten before it is read. **Latching inScale:** if input==output, writing to outputSymInt32QC->scale would also change inputSymInt32QC->scale (same struct). Latching saves it first. **Never saturates** because absMax/qMax always fits within [qMin, qMax].

requantSymInt32TensorToScale() — Pre-Set Scale (One-Pass)

```
float inScale = inputSymInt32QC->scale;
float targetScale = outputSymInt32QC->scale;
// Caller must set targetScale BEFORE calling!
if (!(targetScale > 0.f)) {
PRINT_ERROR(...); exit(1);
}
for (size_t i = 0; i < n; i++) {
outputInt32[i] = roundByMode(
clamp((inputInt32[i]*inScale)/targetScale,
qMin, qMax),
roundingMode);
}
```

When to use: when the target scale is known in advance (e.g. from a calibration pass). Faster — only one loop. **Saturates by design:** values outside [qMin,qMax] clamp to the boundary. This is correct per the Deutel 2025 paper (Eq. 4 analog). **Guard:** !(targetScale > 0.f) is NaN-robust — it catches both negative and NaN values. **NOT in conversionMatrix** — call directly when needed.

convertSymInt32TensorToAsymTensor()

`// Dequantize to float first`
`inputAsFloat[i] = inScale *`
`(float)inputAsInt32[i];`
`// Then quantize to asymmetric`
`outputScale = (max-min) / qMax;`
`zeroPoint = roundByMode(min/outputScale, rm);`
`outputInt[i] = clamp(x/outputScale-zeroPoint, 0,`
`qMax);`
`byteConversion(..., 32, ..., qBits, n);`
	Two-stage: first dequantize SYM_INT32 → float values, then re-quantize to asymmetric format. Bridges the symmetric and asymmetric worlds.

7 — ASYM Conversions

Asymmetric tensors pack N-bit values (e.g. uint8) with a scale and a zeroPoint. Real value = (quantized_value + zeroPoint) × scale.

convertAsymTensorToInt32Tensor()

`byteConversion(inputTensor->data, qBits,`
`dataOut, 32, n); // unpack to int32`
`for each i:`
`outputElements[i] += zeroPoint;`
	Unpacks the N-bit values to full int32, then adds zeroPoint. byteConversion handles the bit-width expansion (e.g. 8-bit → 32-bit).

convertAsymTensorToFloatTensor()

`byteConversion(inputTensor->data, qBits,`
`(uint8_t*)inputInt, 32, n);`
`outputElements[i] =`
`((float)inputInt[i] + (float)zeroPoint)`
`* asymQConfig->scale;`
	Full dequantization: unpack bits → int32 → add zeroPoint → multiply by scale → float. Formula: real = (q + zeroPoint) × scale.

convertAsymTensorToSymInt32Tensor()

`byteConversion(inputTensor->data,`
`bitsPerInputElement,`
`(uint8_t*)inputAsInt32, 32, n);`
`for each i:`
`inputAsInt32[i] += zeroPoint;`
`memcpy(outputTensor->data, inputAsInt32, n*4);`
`outputSymInt32QConfig->scale =`
`inputAsymQConfig->scale;`
	Transfers the asymmetric scale into the SYM_INT32 qConfig, and adjusts values by the zeroPoint so that 0-centered representation is preserved.

8 — The 6×6 Conversion Matrix (Dispatch Table)

Instead of a giant if-else chain, the code uses a 2D array of function pointers. Row = input type, column = output type. The value at [row][col] is the function to call.

```
_Static_assert(BOOL + 1 == 6,
"extend conversionMatrix when adding qtype_t
entries");
```

Compile-time safety check: if someone adds a new enum value (e.g. FLOAT16 = 6), the array size 6 is no longer big enough. This assert catches it at compile time — the build fails with the message shown, before any bugs happen at runtime.

The matrix uses designated initializers:

```
conversionFunction_t conversionMatrix[6][6] = {
[INT32] = {
[INT32] = NULL, // same-type: handled by convertTensorsWithSameType
[FLOAT32] = convertInt32TensorToFloatTensor,
[SYM_INT32]= convertInt32TensorToSymInt32Tensor,
[SYM] = unsupportedConversionTypes,
[ASYM] = convertInt32TensorToAsymTensor,
[BOOL] = unsupportedConversionTypes,
},
[FLOAT32] = {
[INT32] = convertFloatTensorToInt32Tensor,
[FLOAT32] = NULL,
[SYM_INT32]= convertFloatTensorToSymInt32Tensor,
...
},
[SYM_INT32] = {
[INT32] = extractInt32TensorFromSymInt32Tensor,
[FLOAT32] = convertSymInt32TensorToFloat32Tensor,
[SYM_INT32]= requantSymInt32Tensor,
[ASYM] = convertSymInt32TensorToAsymTensor,
...
},
... (SYM/ASYM/BOOL rows omitted for brevity)
};
```

Conversion Matrix — supported (✓) vs unsupported (✗) vs same-type (=):

FROM \ TO	INT32	FLOAT32	SYM_INT32	SYM	ASYM	BOOL
INT32	=	✓	✓	✗	✓	✗
FLOAT32	✓	=	✓	✓	✓	✗
SYM_INT32	✓	✓	✓ (requant)	✗	✓	✗
SYM	✗	✗	✗	=	✗	✗
ASYM	✓	✓	✓	✗	=	✗
BOOL	✗	✗	✗	✗	✗	=

■ **Designated initializers:** [INT32] = {...} uses the enum integer value (e.g. 0) as the array index directly. This is C99 syntax that makes the code self-documenting — you can see which row is INT32 without counting.

9 — convertTensorsWithSameType()

When input and output have the SAME quantization type, no numeric conversion is needed. The data just needs to be copied, along with the quantization parameters.

```
static void convertTensorsWithSameType(
tensor_t *inputTensor,
tensor_t *outputTensor,
qtype_t qType) {
size_t n =
calcNumberOfElementsByTensor(inputTensor);
size_t nb = calcNumberOfBytesForData(
inputTensor->quantization, n);
memmove(outputTensor->data, inputTensor->data,
nb);
// memmove, not memcpy — safe if buffers overlap
switch (qType) {
case SYM_INT32:
outQC->scale = inQC->scale; break;
case ASYM:
outQC->scale = inQC->scale;
outQC->zeroPoint = inQC->zeroPoint; break;
default: break;
}
}
```

memmove vs memcpy: memmove is safe even when src and dst overlap (when input==output buffer). memcpy has undefined behaviour on overlapping buffers. **static:** only visible within TensorConversion.c — internal helper, not exported. **Q-config copy:** for types that carry scale/zeroPoint, copy those fields too so the output tensor is fully self-contained. FLOAT32, INT32, BOOL, SYM hit the default case — no extra parameters needed.

10 — convertTensor() — The Main Entry Point

This is the only function you normally call. It decides whether to use the fast same-type path or to look up the right converter in the matrix.

```
void convertTensor(tensor_t *inputTensor,
tensor_t *outputTensor) {
qtype_t inputDType =
inputTensor->quantization->type;
qtype_t outputDType =
outputTensor->quantization->type;
if (inputDType == outputDType) {
convertTensorsWithSameType(
inputTensor, outputTensor, inputDType);
} else {
conversionFunction_t fn =
conversionMatrix[inputDType][outputDType];
if (fn == NULL) {
PRINT_ERROR("No conversion function...");
exit(1);
}
fn(inputTensor, outputTensor);
}
}
```

Step 1: Read both types. **Step 2:** Same type? → fast memmove path. **Step 3:** Different types → look up conversionMatrix[inputDType][outputDType]. **Step 4:** NULL check — if no converter is registered, print an error and exit cleanly rather than calling NULL (which would segfault). **Step 5:** Call fn(inputTensor, outputTensor) — invokes whichever function was stored in the table. The caller never needs to know which of the 13 converter functions exists.

★ **Flow example:** You have a FLOAT32 weight tensor and want SYM_INT32. Call convertTensor(floatTensor, symInt32Tensor). inputDType=FLOAT32(1), outputDType=SYM_INT32(2). conversionMatrix[1][2] = convertFloatTensorToSymInt32Tensor. That function is called automatically.

11 — quantTypeToString() and unsupportedConversionTypes()

quantTypeToString()

```
char *quantTypeToString(qtype_t t) {
switch (t) {
case INT32: return "INT32";
case FLOAT32: return "FLOAT32";
case SYM_INT32:return "SYMINT32";
case SYM: return "SYM";
case ASYM: return "ASYM";
case BOOL: return "BOOL";
default: return "UNKNOWN";
}
}
```

Converts enum → string for error messages. Returns string literals (compile-time constants) so no malloc needed. Used by PRINT_ERROR to say things like "Conversion from FLOAT32 to BOOL is not supported".

unsupportedConversionTypes()

```
void unsupportedConversionTypes(
tensor_t *inputTensor,
tensor_t *outputTensor) {
PRINT_ERROR(
"Conversion from %s to %s is not supported",
quantTypeToString(inputQType),
quantTypeToString(outputQType));
exit(1);
}
```

This function has the SAME signature as conversionFunction_t, so it can be stored in the matrix. When a caller requests an unsupported conversion (e.g. FLOAT32→BOOL), this function is called and immediately prints a clear error and exits — much better than a NULL dereference.

12 — Function Reference Summary

Function	What it does	How called
`zeroTensorData(t)`	Fills t->data with zero bytes	Internal helper
`copyDimsAndSparsityToTensor(in, out)`	Copies shape pointer + sparsity from in→out	Internal helper
`convertInt32TensorToFloatTensor(in, out)`	int32 cast to float, element-wise	Matrix
`convertInt32TensorToSymInt32Tensor(in, out)`	memcpy data, set scale=1	Matrix
`convertInt32TensorToAsymTensor(in, out)`	Compute scale+zeroPoint, quantize to N bits	Matrix
`convertFloatTensorToInt32Tensor(in, out)`	Truncate float to int32	Matrix
`convertFloatTensorToSymInt32Tensor(in, out)`	absMax → scale → quantize to int32 mantissas	Matrix
`convertFloatTensorToSymTensor(in, out)`	N-bit symmetric quantization (placeholder)	Matrix
`convertFloatTensorToAsymTensor(in, out)`	N-bit asymmetric quantization from float	Matrix
`extractInt32TensorFromSymInt32Tensor(in, out)`	Copy mantissas, drop scale	Matrix
`convertSymInt32TensorToFloat32Tensor(in, out)`	Dequantize: mantissa × scale	Matrix
`requantSymInt32Tensor(in, out)`	SYM_INT32 → SYM_INT32, fresh scale (2-pass)	Matrix + Direct
`requantSymInt32TensorToScale(in, out)`	SYM_INT32 → SYM_INT32, preset scale (1-pass)	Direct only
`convertSymInt32TensorToAsymTensor(in, out)`	Dequantize then re-quantize to asymmetric	Matrix
`convertAsymTensorToInt32Tensor(in, out)`	Unpack bits + add zeroPoint → int32	Matrix
`convertAsymTensorToFloatTensor(in, out)`	Unpack bits → (q+zp)×scale → float	Matrix
`convertAsymTensorToSymInt32Tensor(in, out)`	Unpack bits + add zeroPoint → SYM_INT32	Matrix

`convertTensorsWithSameType(in, out, t)`	memmove data, copy Q-params	Internal
`convertTensor(in, out)`	Main entry point — auto-dispatch	Public API
`quantTypeToString(t)`	enum → string for error messages	Public API
`unsupportedConversionTypes(in, out)`	Error function stored in matrix for unsupported pairs	Sentinel

13 — Quick Cheat Sheet

Convert any tensor	convertTensor(src, dst) — always use this
Same-type copy	convertTensor handles it via memmove
Float → quantized int32	convertFloatTensorToSymInt32Tensor (most common for FQT)
Quantized int32 → float	convertSymInt32TensorToFloat32Tensor (dequantize)
Rescale SYM_INT32 (dynamic)	requantSymInt32Tensor — computes fresh scale
Rescale SYM_INT32 (preset)	requantSymInt32TensorToScale — caller sets scale first
Function pointer type	conversionFunction_t = void (*)(tensor_t*, tensor_t*)
Dispatch mechanism	conversionMatrix[inputType][outputType](in, out)
Compile-time safety	_Static_assert(BOOL+1==6, ...) — catches missing entries
Unsupported pairs	unsupportedConversionTypes() — prints error and exits
Dequantize formula	float = mantissa × scale (SYM_INT32 → FLOAT32)
Quantize formula	mantissa = clamp(float / scale, qMin, qMax) (FLOAT32 → SYM_INT32)
Asymmetric formula	real = (quantized + zeroPoint) × scale